Tutorial: De LLMs a Agentes Autónomos y Máquinas de Estados (FSM)

Serie: *Guía Práctica de Arquitectura y Desarrollo de Agentes de IA*
Basado en: *AI Agents: The Definitive Guide* (Capítulo 1) por Nicole Koenigstein (O'Reilly Media)
Referencia: 0642572247775 / ch01.html#FSM | **Enfoque:** Práctico, Arquitectónico y Código Comentado

1. Introducción: La Gran Transición de Modelos Estáticos a Agentes

Durante los primeros años de la Inteligencia Artificial Generativa, la interacción con los Modelos de Lenguaje Grande (LLMs) se limitó a un paradigma puramente **estático y sin estado (*stateless*)**: el usuario enviaba un prompt, y el modelo predecía la secuencia más probable de tokens (*Next-Token Prediction*).

En entornos de producción, las tareas reales exigen:
1. **Acceso dinámico a información externa** (consultar bases de datos, APIs, documentos).
2. **Ejecución de acciones con efectos secundarios** (modificar código, invocar servicios externos).
3. **Razonamiento en múltiples etapas** (planificar pasos, verificar resultados intermedios y corregir errores).

> **¿Qué define formalmente a un Agente de IA?**
>
> Un **Agente de IA** es un sistema de software que envuelve a un modelo de razonamiento dentro de un bucle de control dinámico, dotándolo de **Percepción** (observación del entorno), **Memoria** (retención a corto y largo plazo), **Planificación** (descomposición de metas) y **Acción** (actuadores y herramientas).

2. Los 4 Pilares de la Arquitectura de un Agente

Pilar	Responsabilidad Técnica	Retos en Producción
1. Núcleo (LLM)	Inferencia, comprensión contextual y selección de intenciones.	Latencia, coste por token, no-determinismo y alucinaciones.
2. Memoria	Historial conversacional y recuperación semántica (RAG).	Desbordamiento de contexto (*Context Drift*), pérdida de estado intermedio.
3. Planificación	Generación de planes paso a paso y replanificación.	Bucles infinitos (*Infinite Loops*), degradación en planes largos.
4. Herramientas	Ejecución de funciones externas y parseo de respuestas.	Errores de tipado, fallos de red y riesgos de inyección.

3. El Problema del No-Determinismo: Por qué los Bucles Abiertos Fracasan

El error común al diseñar agentes es implementar un bucle `while True` no regulado donde se le pide al LLM: «*Decide qué hacer a continuación hasta terminar*». Este enfoque de **bucle abierto (*Open Loop*)** colapsa en entornos reales debido a:

- **Deriva de Estado (*State Drift*):** A medida que el contexto crece, el LLM olvida el objetivo original o repite llamadas redundantes.
- **Bucles de Fallo Infinitos:** Ante errores en APIs o argumentos inválidos, el modelo insiste en llamar a la misma función con los mismos parámetros erróneos.
- **Falta de Transiciones Predecibles:** Imposibilidad de auditar formalmente qué caminos puede tomar la aplicación en producción.

4. La Solución: Máquinas de Estados Finitos (FSM)

Para dotar al sistema de predictibilidad y robustez, se aplica el modelo clásico de **Máquinas de Estados Finitos (FSM)**:

- **Estados (S):** Modos bien delimitados (`IDLE`, `PLANNING`, `EXECUTING_TOOL`, `VALIDATING`, `ERROR_RECOVERY`, `DONE`).
- **Eventos (E):** Señales que provocan una evaluación de cambio (`RECIBIR_TAREA`, `FALLO_HERRAMIENTA`, etc.).
- **Guardas (G):** Condiciones deterministas que autorizan la transición (ej. `reintentos < max_retries`).

- **Transiciones (δ):** Reglas explícitas que mueven al agente de un estado a otro de manera controlada.

5. Implementación Práctica en Python y Análisis Paso a Paso

A continuación se detalla la implementación completa con pasos numerados y su correspondiente análisis técnico:

5.1. Código Fuente Completo y Comentado

```python
import time
from enum import Enum, auto
from typing import Dict, Any, Optional, List

# ============================================================================
# [PASO 1] DECLARACIÓN DE ESTADOS Y EVENTOS FORMALES (ENUMS)
# ============================================================================
class AgentState(Enum):
    IDLE = auto()                 # Reposo: Esperando asignación de objetivo
    PLANNING = auto()             # Planificación: LLM genera/revisa el plan de acción
    EXECUTING_TOOL = auto()       # Ejecución: Llamada a APIs, bases de datos o scripts
    VALIDATING = auto()           # Validación: Verificación de calidad y consistencia
    ERROR_RECOVERY = auto()       # Recuperación: Manejo controlado de excepciones
    DONE = auto()                 # Terminal Exitoso: Tarea completada y verificada
    TERMINATED_FAILED = auto()    # Terminal Fallido: Límite de reintentos agotado

class AgentEvent(Enum):
    START = auto()
    PLAN_READY = auto()
    TOOL_SUCCESS = auto()
    TOOL_FAILED = auto()
    VALIDATION_PASSED = auto()
    NEED_MORE_STEPS = auto()
    RETRY_AUTHORIZED = auto()
    MAX_RETRIES_REACHED = auto()

# ============================================================================
# [PASO 2] ESTRUCTURA DE CONTEXTO Y PERSISTENCIA DE MEMORIA (AgentContext)
# ============================================================================
class AgentContext:
    def __init__(self, task: str, max_retries: int = 3):
        self.task: str = task                         # Meta global asignada al agente
        self.max_retries: int = max_retries         # Guarda: Límite máximo de fallos tolerados
        self.retry_count: int = 0                     # Contador de intentos fallidos
        self.plan: List[str] = []                     # Lista ordenada de subtareas
        self.current_step_index: int = 0              # Índice del paso en ejecución
        self.execution_results: Dict[str, Any] = {} # Almacén de salidas intermedias
        self.last_error: Optional[str] = None       # Registro del último fallo capturado
        self.final_output: Optional[str] = None     # Resultado final sintetizado
        self.history_log: List[str] = []              # Bitácora cronológica de auditoría

    def log(self, message: str) -> None:
        entry = f"[{time.strftime('%H:%M:%S')}] {message}"
        self.history_log.append(entry)
        print(f"   📌 {entry}")

# ============================================================================
# [PASO 3] CLASE PRINCIPAL DE LA FSM Y AUDITORÍA DE TRANSICIONES (FSMAgent)
# ============================================================================
class FSMAgent:
    def __init__(self, context: AgentContext):
        self.context = context
        self.state = AgentState.IDLE
        self.context.log(f"Agente inicializado en estado: {self.state.name}")

    def transition_to(self, new_state: AgentState, reason: str = "") -> None:
        self.context.log(f"TRANSICIÓN: {self.state.name} -> {new_state.name} | Motivo: {reason}")
        self.state = new_state

    # ============================================================================
    # [PASO 4] CONTROLADORES DE ESTADO ESPECIALIZADOS (STATE HANDLERS)
    # ============================================================================
    def handle_idle(self) -> None:
        self.context.log(f"Tarea asignada: \"{self.context.task}\"")
```

```
          self.transition_to(AgentState.PLANNING, "Iniciando análisis y descomposición")

     def handle_planning(self) -> None:
          self.context.log("Cerebro (LLM) formulando el plan de acción estructurado...")
          if "financiero" in self.context.task.lower():
               self.context.plan = ["fetch_financial_data", "calculate_ebitda_margins",
"generate_executive_report"]
               self.context.current_step_index = 0
               self.transition_to(AgentState.EXECUTING_TOOL, "Plan validado; procediendo a ejecutar paso
1")
          else:
               self.context.last_error = "Dominio de tarea no soportado."
               self.transition_to(AgentState.ERROR_RECOVERY, "Fallo en generación de plan")

     def handle_executing_tool(self) -> None:
          step = self.context.plan[self.context.current_step_index]
          self.context.log(f"Ejecutando herramienta ({self.context.current_step_index + 1}/
{len(self.context.plan)}): [{step}]")
          try:
               if step == "fetch_financial_data":
                    if self.context.retry_count == 0 and "forzar_fallo" in self.context.task:
                         raise ConnectionError("Timeout 504: El servidor de datos no respondió.")
                    self.context.execution_results["raw_data"] = {"ingresos": 1500000, "gastos": 950000}
                    self.transition_to(AgentState.VALIDATING, "Datos obtenidos; enviando a validación")
               elif step == "calculate_ebitda_margins":
                    raw = self.context.execution_results["raw_data"]
                    ebitda = raw["ingresos"] - raw["gastos"]
                    margen = (ebitda / raw["ingresos"]) * 100
                    self.context.execution_results["metrics"] = {"ebitda": ebitda, "margen": margen}
                    self.transition_to(AgentState.VALIDATING, "Cálculos completados")
               elif step == "generate_executive_report":
                    m = self.context.execution_results["metrics"]
                    self.context.final_output = f"INFORME Q3: EBITDA=${m['ebitda']:,.2f} USD |
Margen={m['margen']:.2f}%"
                    self.transition_to(AgentState.VALIDATING, "Informe redactado")
          except Exception as e:
               self.context.last_error = str(e)
               self.context.log(f"❌ Error en herramienta: {e}")
               self.transition_to(AgentState.ERROR_RECOVERY, "Excepción capturada en ejecución")

     def handle_validating(self) -> None:
          step = self.context.plan[self.context.current_step_index]
          if step in ["fetch_financial_data", "calculate_ebitda_margins"]:
               self.context.current_step_index += 1
               if self.context.current_step_index < len(self.context.plan):
                    self.transition_to(AgentState.EXECUTING_TOOL, "Paso validado; avanzando")
               else:
                    self.transition_to(AgentState.DONE, "Todos los pasos completados")
          elif step == "generate_executive_report":
               if self.context.final_output:
                    self.transition_to(AgentState.DONE, "Validación final satisfactoria")
               else:
                    self.context.last_error = "Salida final corrupta"
                    self.transition_to(AgentState.ERROR_RECOVERY, "Fallo en validación final")

     def handle_error_recovery(self) -> None:
          self.context.retry_count += 1
          self.context.log(f"Recuperación ante desastres (Intento {self.context.retry_count}/
{self.context.max_retries})...")
          if self.context.retry_count <= self.context.max_retries:
               time.sleep(0.5 * (2 ** (self.context.retry_count - 1)))
               self.transition_to(AgentState.EXECUTING_TOOL, "Reintento autorizado")
          else:
               self.context.log("❌ Límite de reintentos alcanzado. Abortando misión de forma segura.")
               self.transition_to(AgentState.TERMINATED_FAILED, "Límite de fallos superado")

     # ==========================================================================
     # [PASO 5] BUCLE PRINCIPAL DE DESPACHO Y CONTROL (RUN LOOP)
     # ==========================================================================
     def run(self) -> Dict[str, Any]:
          self.context.log("Iniciando bucle de control del agente...")
```

```
        while self.state not in [AgentState.DONE, AgentState.TERMINATED_FAILED]:
            if self.state == AgentState.IDLE: self.handle_idle()
            elif self.state == AgentState.PLANNING: self.handle_planning()
            elif self.state == AgentState.EXECUTING_TOOL: self.handle_executing_tool()
            elif self.state == AgentState.VALIDATING: self.handle_validating()
            elif self.state == AgentState.ERROR_RECOVERY: self.handle_error_recovery()

        success = (self.state == AgentState.DONE)
        return {"success": success, "state": self.state.name, "output": self.context.final_output}

# ============================================================================
# [PASO 6] PUNTO DE ENTRADA Y DEMOSTRACIÓN DE RESILIENCIA OPERATIVA
# ============================================================================
if __name__ == "__main__":
    ctx = AgentContext("Generar informe financiero Q3 (forzar_fallo_red_inicial)", max_retries=3)
    agent = FSMAgent(ctx)
    res = agent.run()
    print(f"\nResultado: {res}")
```

5.2. Desglose Técnico y Análisis Paso a Paso

1. **Paso 1 (Enums Tipados):** Define un conjunto cerrado de estados finitos y eventos. Elimina strings mágicos y permite validación estática de tipos.
2. **Paso 2 (Contenedor de Estado - `AgentContext`):** Desacopla la memoria y los datos de la lógica del agente. Permite serializar el estado para crear *checkpoints* y reanudar sesiones tras caídas.
3. **Paso 3 (Trazabilidad - `transition_to`):** Registra cronológicamente cada cambio de estado junto a su justificación técnica, indispensable para auditorías en entornos de producción.
4. **Paso 4 (Aislamiento de Responsabilidades en Handlers):**
   - `4.1` valida la tarea de entrada.
   - `4.2` descompone la meta mediante el LLM.
   - `4.3` ejecuta herramientas aislando excepciones bajo `try/except`.
   - `4.4` aplica guardas de calidad a las salidas intermedias.
   - `4.5` aplica la guarda determinista de reintentos (`retry_count <= max_retries`) con retroceso exponencial (*backoff*), erradicando bucles infinitos.
5. **Paso 5 (Bucle Central Determinista):** El ciclo `while` solo finaliza cuando la máquina alcanza un estado terminal (`DONE` o `TERMINATED_FAILED`), garantizando un tiempo y coste acotados.
6. **Paso 6 (Resiliencia Comprobada):** Ante un fallo de red 504 simulado, el agente entra en `ERROR_RECOVERY`, aplica pausa y reintenta con éxito hasta concluir en `DONE`.

6. Evolución a Grafos de Estado (LangGraph) y Conclusiones

En sistemas agénticos avanzados, el modelo de FSM se formaliza mediante **Grafos de Estado (*State Graphs*)**:

1. **Nodos:** Funciones que transforman el estado del sistema.
2. **Bordes Condicionales:** Enrutamiento lógico basado en la salida del LLM o de las herramientas.
3. **Puntos de Control (*Checkpoints*):** Persistencia del estado completo en cada nodo para permitir reanudación tras fallos (*Time Travel / Resumption*).
4. **Humano en el Bucle (*Human-in-the-Loop*):** Modelar la autorización humana como un estado bloqueante antes de ejecutar acciones de alto impacto.

Próximo Paso: Capítulo 2

Architectures and Patterns: Planning, Reactivity, and Multi-Agent Systems — Patrones avanzados de planificación (ReAct, Reflexion, Tree of Thoughts) y arquitecturas multi-agente distribuidas.